

Lab 1: Simulation of Communication Systems

ENG3014 Communication Systems 3

Kartike
Sarah Mann
October 2026

Contents

1 Introduction **3**

1.1 Setup 3

1.2 Notation 3

2 Task 1: Creating Arrays and Plotting Data **4**

2.1 Theory: from a continuous wave to samples 4

2.2 Code and how it works 5

2.3 Results 7

2.4 Why the plots look like this 8

3 Task 2: Modulation of a Carrier Wave with Pure Tones **8**

3.1 Theory: amplitude modulation 8

3.2 Code and how it works 9

3.3 Results 11

3.4 Why the plots look like this 11

4 Task 3: Discrete Fourier Transform (DFT) **12**

4.1 Theory 12

4.2 Code and how it works 13

4.3 Results 15

4.4 Why the plots look like this 17

5 Task 4: Signal Filtering **18**

5.1 Theory: filtering by multiplying in k-space 18

5.2 Code and how it works 19

5.3 Results 21

5.4 Why the plots look like this 21

6 Conclusion **22**

1 Introduction

This report documents the attempt at each of the four tasks in the ENG3014 simulation lab. The aim of the lab is to practise the basic building blocks used when a communication system is simulated in code: creating arrays of sampled data, amplitude modulation, the discrete Fourier transform (DFT) and filtering in the frequency domain. For every task the report shows the code, explains what each part of it does, presents the plots it produces and explains why the plots look the way they do.

The goals of the four tasks were to:

- represent a continuous sine wave as an array of samples, and see how the sample frequency and the number of samples change what is stored (Task 1);
- build an amplitude modulated (AM) signal from a carrier and a pure-tone message (Task 2);
- write a DFT directly from its formula, scale its axis to hertz, explore how the signal and sampling parameters change the spectrum, and compare it with NumPy's fast Fourier transform (Task 3);
- filter the AM signal by multiplying its spectrum by a square band-pass window, then return to the time domain with the inverse FFT (Task 4).

1.1 Setup

The code was written in Python 3.9 in a Jupyter notebook in VS Code, using NumPy 2.0 for arrays, maths and the FFT, and Matplotlib 3.9 for plotting. The notebook has five cells: one for the imports (Listing 1) and one for each task. Each task cell starts with a line such as `"""-----TASK 1-----"""`. This is a string on its own line, which Python evaluates and then discards, so it does nothing except act as a visible label. Later cells reuse functions and variables from earlier ones (for example, Task 3 calls the AM function written in Task 2), so the cells must be run in order.

Listing 1: Imports, first cell of the notebook.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

`np` and `plt` are the short names used for NumPy and Matplotlib's plotting module throughout the notebook.

1.2 Notation

The symbols follow the lab sheet. Table 1 links each symbol to the variable that holds it in the code, so the equations in each section can be matched to the lines that compute them.

Table 1: Symbols used in this report and the variables that hold them in the code.

Symbol	Code variable	Meaning
a	`a`	amplitude of the Task 1 sine wave
ν	`hz`	frequency of the Task 1 sine wave (Hz)
f_s	`sf` (Task 1), `fs` (Tasks 2–4)	sample frequency (samples per second)
N	`N`	number of samples
$t_n = n/f_\mathrm{s}$	`t`	time at which sample n is taken
$T = N/f_\mathrm{s}$	`duration`	record length (time covered by the samples)
A	`A`	signal amplitude
DC	`DC`	DC offset added to the message
m	`m`	modulation amplitude
ν_m	`nu_m`	modulation (message) frequency (Hz)
ν_c	`nu_c`	carrier frequency (Hz)
F_k	`F`	DFT value in frequency bin k
$\nu_k = kf_\mathrm{s}/N$	`freqs`	frequency of bin k (Hz)
$\Delta f = f_\mathrm{s}/N$		spacing between bins (frequency resolution)

Each task section follows the same order: the theory needed, the code with a line-by-line explanation, the results, and an explanation of why the plots look the way they do.

2 Task 1: Creating Arrays and Plotting Data

2.1 Theory: from a continuous wave to samples

The wave to be represented is

$$f(t) = a\sin(2\pi\nu t). \tag{1}$$

A computer cannot store $f(t)$ at every instant. It stores N numbers taken at equal intervals of $1/f_\mathrm{s}$, where f_s is the sample frequency. Sample n (with $n = 0, 1, \ldots, N-1$) is taken at time t_n and has value f_n:

$$t_n = \frac{n}{f_\mathrm{s}}, \qquad f_n = a\sin\left(2\pi\nu\,\frac{n}{f_\mathrm{s}}\right). \tag{2}$$

Together the samples cover a record of length

$$T = \frac{N}{f_\mathrm{s}}. \tag{3}$$

These are two separate choices. The sample frequency decides how many samples fall within each cycle of the wave (f_s/ν), so it decides how finely the shape of one cycle is captured. The number of samples, together with f_s, decides how long the record lasts.

Nyquist frequency and aliasing. The highest frequency that a set of samples can hold is the Nyquist frequency,

$$f_\mathrm{N} = 0.5\,f_\mathrm{s}, \tag{4}$$

so the wave must satisfy $\nu < f_\mathrm{s}/2$: there must be more than two samples in every cycle. If ν is above f_N, the samples are identical to those of a lower frequency, called an alias. This follows from Equation (2): adding a whole number of cycles ($2\pi n$) to the angle of a sine does not change its value, so

$$\sin\left(2\pi\nu\,\frac{n}{f_\mathrm{s}}\right) = \sin\left(2\pi(\nu - f_\mathrm{s})\,\frac{n}{f_\mathrm{s}}\right). \tag{5}$$

A wave at ν therefore produces exactly the same samples as a wave at $\nu - f_s$. For $f_s/2 < \nu < f_s$ the wave appears at $|\nu - f_s|$: its frequency is folded back about f_N.

2.2 Code and how it works

Listing 2: Task 1, lines 1–19: the function that samples the sine wave.

```
1  """-----TASK 1-----"""
2
3  def sine_wave(a, hz, sf, N):
4      """
5      Discrete samples of f(t) = a*sin(2*pi*hz*t).
6
7      a  : amplitude
8      hz : wave frequency in Hz
9      sf : sample frequency in Hz (samples per second)
10     N  : number of samples
11     Returns (t, f): the sample times and the sampled values.
12     """
13     # Sample n is taken at time t_n = n / sf, for n = 0, 1, ..., N-1.
14     # np.arange(N) gives [0, 1, ..., N-1]; dividing by sf converts to seconds.
15     t = np.arange(N) / sf
16
17     # Evaluate the wave at every sample time at once (NumPy works element-wise).
18     f = a * np.sin(2 * np.pi * hz * t)
19     return t, f
```

- **Line 3** defines `sine_wave`, which takes the quantities in Equations (1) and (2): the amplitude `a`, the wave frequency `hz` (ν), the sample frequency `sf` (f_s) and the number of samples `N`.
- **Line 15** builds the sample times. `np.arange(N)` creates the array $[0, 1, 2, \ldots, N-1]$, which are the sample numbers n. Dividing the array by `sf` divides every element at once, giving $t_n = n/f_s$ from Equation (2).
- **Line 18** evaluates Equation (1) at every sample time in one step. NumPy applies `np.sin` to each element of `t`, so `f` is an array of the N values f_n: the list of values the task asks for. No Python loop is needed.
- **Line 19** returns both arrays. Returning `t` as well as `f` means the plots can use real time in seconds on the x-axis rather than sample numbers.

Listing 3: Task 1, lines 22–36: the wave, the sampling cases and the reference wave.

```
22 # --- Parameters for the wave itself ---
23 a = 1.0      # amplitude
24 hz = 5.0     # 5 Hz sine wave
25
26 # --- Different (sample frequency, number of samples) cases to compare ---
27 cases = [(100, 100),   # sf well above the wave frequency
28          (20, 20),     # fewer samples per cycle
29          (8, 8),       # sf below 2*hz: watch what happens
30          (100, 300)]   # more samples: longer time window
31
32 # A "continuous" reference: sampled very finely so it looks smooth.
33 # It must last as long as the longest case (sf = 100, N = 300 lasts 3 s),
34 # otherwise the grey line stops early on that plot.
35 longest = max(N / sf for sf, N in cases)
36 t_ref, f_ref = sine_wave(a, hz, sf=10000, N=int(10000 * longest))
```

- **Lines 23–24** fix the wave at $a = 1$ and $\nu = 5$ Hz for every case, so the only thing that changes between plots is the sampling.
- **Lines 27–30** list the four (f_s, N) pairs to compare. They are summarised in Table 2: the first three lower f_s while keeping $T = 1$ s, and the last raises N while keeping f_s the same as the first.
- **Lines 35–36** make a reference wave sampled at 10 kHz, which is 2000 samples per cycle of the 5 Hz wave. That is so fine that the joined points look continuous, so it stands in for the true $f(t)$. Line 35 works out the longest record among the cases (it calculates `N / sf` for every pair and keeps the largest, 3 s), and line 36 makes the reference that long. An earlier version used a fixed 10 000 samples (1 s), so the reference stopped a third of the way along the 3 s plot; line 35 fixes this.

Table 2: The four sampling cases for the 5 Hz wave.

Plot	f_s (Hz)	N	$T = N/f_s$ (s)	Samples per cycle, f_s/ν
(a)	100	100	1	20
(b)	20	20	1	4
(c)	8	8	1	1.6 (below Nyquist)
(d)	100	300	3	20

Listing 4: Task 1, lines 38–58: plotting each case.

```
38 # One subplot per case, stacked vertically.
39 fig, axes = plt.subplots(len(cases), 1, figsize=(8, 10))
40
41 for ax, (sf, N) in zip(axes, cases):
42     t, f = sine_wave(a, hz, sf, N)
43     duration = N / sf   # total time covered by the samples
44
45     # Grey line = the true wave, over the same time window
46     mask = t_ref < duration
47     ax.plot(t_ref[mask], f_ref[mask], color="lightgrey", label="true wave")
48
49     # Dots joined by lines = what the computer actually stores
50     ax.plot(t, f, "o-", markersize=3, label="samples")
51
52     ax.set_title(f"sf = {sf} Hz, N = {N}  (duration {duration:.2f} s)")
53     ax.set_xlabel("Time (s)")
54     ax.set_ylabel("f(t)")
55     ax.legend(loc="upper right")
56
57 plt.tight_layout()   # stops titles and labels overlapping
58 plt.show()
```

- **Line 39** creates one figure with four plots stacked in a single column. `axes` holds one plot area per case, and `figsize` is the figure size in inches.
- **Line 41** uses `zip` to pair the first plot area with the first case, the second with the second, and so on. Each pass of the loop unpacks a pair into `sf` and `N`.
- **Lines 42–43** compute the samples for this case and the record length $T = N/f_s$ from Equation (3).
- **Lines 46–47** trim the reference wave to the same time window. `t_ref < duration` compares every reference time with T and gives an array of `True`/`False` values (a mask). Indexing with

it, `t_ref[mask]`, keeps only the elements where the mask is `True`. The trimmed reference is drawn in light grey as the true wave.

- **Line 50** plots the samples with the format `"o-"`: a dot at each sample, joined by straight lines. The dots are the only data the computer stores. The straight lines are drawn by Matplotlib to join them, which is why a badly sampled wave can look like a different shape.
- **Lines 52–55** add a title (an f-string that inserts `sf`, `N` and the duration to two decimal places), axis labels and a legend.
- **Lines 57–58** space the plots so the labels do not overlap, then display the figure.

2.3 Results

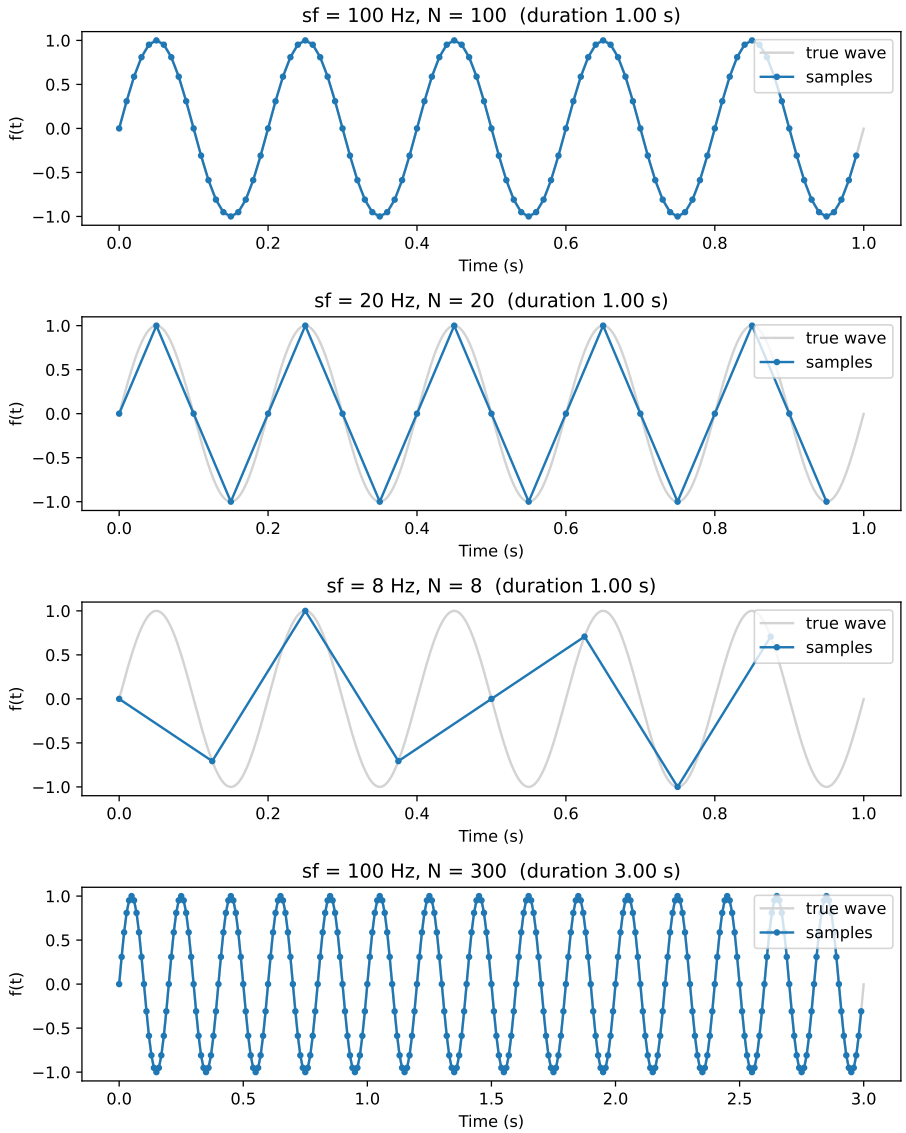


Figure 1: The 5 Hz sine wave sampled with the four cases of Table 2, labelled (a) to (d) from top to bottom. Grey: reference wave sampled at 10 kHz. Blue: the stored samples (dots) joined by straight lines.

2.4 Why the plots look like this

- **(a)** $f_\mathrm{s} = 100$ **Hz,** $N = 100$**.** There are 20 samples per cycle, so the joined dots are almost indistinguishable from the grey reference. The record lasts $T = 1$ s, so it shows exactly 5 cycles.
- **(b)** $f_\mathrm{s} = 20$ **Hz,** $N = 20$**.** There are only 4 samples per cycle. From Equation (2) the samples are $\sin(2\pi \cdot 5\, n/20) = \sin(\pi n/2) = 0, 1, 0, -1, 0, \ldots$, so they land exactly on the zero crossings and the peaks, and the straight lines between them form a triangle wave. The samples themselves are exact and the frequency is still correct (5 peaks in 1 s), because $f_\mathrm{s} = 20$ Hz is above $2\nu = 10$ Hz. Only the joining lines make the shape look wrong.
- **(c)** $f_\mathrm{s} = 8$ **Hz,** $N = 8$**.** Here f_s is below $2\nu = 10$ Hz, so the Nyquist condition is broken ($f_\mathrm{N} = 4$ Hz). The samples are $0, -0.71, 1, -0.71, 0, 0.71, -1, 0.71$, and the joined dots trace a slower wave with 3 cycles in 1 s instead of 5. This is the alias predicted by Equation (5): $\nu - f_\mathrm{s} = 5 - 8 = -3$ Hz, and since $\sin(-x) = -\sin x$, the samples are exactly those of a 3 Hz sine turned upside down. The 5 Hz wave is 1 Hz above f_N, so it folds to 1 Hz below f_N. Every dot still lies on the grey 5 Hz curve: both the 5 Hz wave and the 3 Hz alias pass through the same points, so the samples alone cannot tell them apart. Once the signal has been sampled this slowly, the information about its true frequency is lost.
- **(d)** $f_\mathrm{s} = 100$ **Hz,** $N = 300$**.** The sample frequency is the same as in (a), so each cycle is captured just as well. The only change is that the record now lasts $T = 3$ s and shows 15 cycles. Taking more samples at the same rate gives a longer record, not more detail.

In summary, the sample frequency sets how faithfully each cycle is captured and the highest frequency that can be represented, while N/f_s sets the length of the record. Task 3 shows that the record length matters in the frequency domain too.

3 Task 2: Modulation of a Carrier Wave with Pure Tones

3.1 Theory: amplitude modulation

In amplitude modulation the information (here a pure tone, the message) is carried by changing the amplitude of a high-frequency carrier wave. The signal for this task is

$$y(t) = \big[\mathrm{DC} + m \sin(2\pi\nu_m t)\big]\, A \sin(2\pi\nu_c t). \tag{6}$$

It is the product of two factors. The carrier, $A\sin(2\pi\nu_c t)$, oscillates quickly with a constant amplitude. The bracket changes slowly, because ν_m is much smaller than ν_c. Multiplying them gives a carrier whose amplitude, $A[\mathrm{DC} + m\sin(2\pi\nu_m t)]$, rises and falls with the message. The curve traced by the peaks of the carrier is called the envelope:

$$\pm A\big[\mathrm{DC} + m \sin(2\pi\nu_m t)\big]. \tag{7}$$

The envelope has the same shape as the message, so a receiver can recover the message by following the peaks of the carrier. This is the job of a diode envelope detector.

Why the DC offset is needed. A sine wave is negative for half of every cycle. Without the DC term, the bracket would be negative for half of every message cycle, and multiplying the carrier by a negative number flips it upside down (a 180° change of phase) instead of making it smaller. The DC offset lifts the bracket so that, as long as $m \le \mathrm{DC}$, it never drops below zero

and the envelope is an exact copy of the message. The ratio m/DC is the modulation depth, which is 50% for the baseline used here. If $m > \mathrm{DC}$, the bracket goes negative for part of each message cycle, the outline of the signal follows the size of the bracket, $|\mathrm{DC} + m\sin(2\pi\nu_m t)|$, and it no longer matches the message.

Multiplying two sine waves also creates new frequencies. This is what the spectrum in Task 3 shows.

3.2 Code and how it works

Listing 5: Task 2, lines 1–19: the AM signal function.

```
1  """-----TASK 2-----"""
2  def am_signal(A, DC, m, nu_m, nu_c, fs, N):
3      """
4      Discrete samples of the AM signal
5          y(t) = [DC + m*sin(2*pi*nu_m*t)] * A*sin(2*pi*nu_c*t)
6
7      A    : signal amplitude
8      DC   : DC offset added to the message
9      m    : modulation amplitude
10     nu_m : modulation (message) frequency in Hz
11     nu_c : carrier frequency in Hz
12     fs   : sample frequency in Hz
13     N    : number of samples
14     """
15     t = np.arange(N) / fs                                  # same sample times as Task 1
16     message = DC + m * np.sin(2 * np.pi * nu_m * t)        # slowly varying envelope term
17     carrier = A * np.sin(2 * np.pi * nu_c * t)             # fast carrier
18     y = message * carrier                                  # AM: the message multiplies the
       ↪ carrier
19     return t, y
```

- **Line 2** defines `am_signal`. Its inputs are the symbols of Equation (6) plus the sample frequency `fs` and number of samples `N`. `fs` is the same quantity that was called `sf` in Task 1.
- **Line 15** makes the same sample times as Task 1, $t_n = n/f_\mathrm{s}$ (Equation (2)).
- **Line 16** computes the bracket of Equation (6) at every sample time: the message plus its DC offset.
- **Line 17** computes the carrier at every sample time.
- **Line 18** multiplies the two arrays element by element, so each output sample is (bracket at t_n) $\times$ (carrier at t_n). This single multiplication is the modulation.
- **Line 19** returns the sample times and the AM signal.

Listing 6: Task 2, lines 22–42: the baseline parameters and the plot of two modulation amplitudes.

```
22 # --- Baseline parameters (reused in Tasks 3 and 4) ---
23 A, DC, m = 1.0, 1.0, 0.5
24 nu_m, nu_c = 5.0, 50.0       # 5 Hz message on a 50 Hz carrier
25 fs, N = 1000, 1000           # 1 s of data sampled at 1 kHz
26
27 # Compare two modulation amplitudes: m < DC and m > DC
28 fig, axes = plt.subplots(2, 1, figsize=(8, 6))
29 for ax, m_demo in zip(axes, [0.5, 1.5]):
30     t, y = am_signal(A, DC, m_demo, nu_m, nu_c, fs, N)
```

```
31      envelope = A * (DC + m_demo * np.sin(2 * np.pi * nu_m * t))   # the bracket term, scaled
        ↪ by A
32
33      ax.plot(t, y, label="AM signal")
34      ax.plot(t, envelope, "r--", label="envelope")
35      ax.plot(t, -envelope, "r--")                          # mirror image below the axis
36      ax.set_title(f"m = {m_demo}, DC = {DC}")
37      ax.set_xlabel("Time (s)")
38      ax.set_ylabel("y(t)")
39      ax.legend(loc="upper right")
40
41  plt.tight_layout()
42  plt.show()
```

- **Lines 23–25** set the baseline used here and in Tasks 3 and 4: $A = 1$, DC $= 1$, $m = 0.5$ (50% modulation depth), a 5 Hz message on a 50 Hz carrier, and $f_s = 1000$ Hz with $N = 1000$, so $T = 1$ s. This sample frequency gives 20 samples per carrier cycle, as in Task 1(a), and is far above twice the highest frequency in the signal (55 Hz, as Task 3 shows).
- **Lines 28–29** create two stacked plots and loop over two modulation amplitudes, `m_demo` $= 0.5$ ($m <$ DC) and 1.5 ($m >$ DC). A separate name is used so that the baseline value `m` is not changed.
- **Line 30** generates the AM signal for that modulation amplitude.
- **Line 31** computes the envelope of Equation (7) directly from its formula, not from the signal. It therefore shows where the peaks should be if the message is carried correctly.
- **Lines 33–35** plot the signal, then the envelope as a red dashed line (`"r--"`) and its negative as a mirror image below the axis.
- **Lines 36–42** add titles, labels and a legend, then lay out and display the figure.

3.3 Results

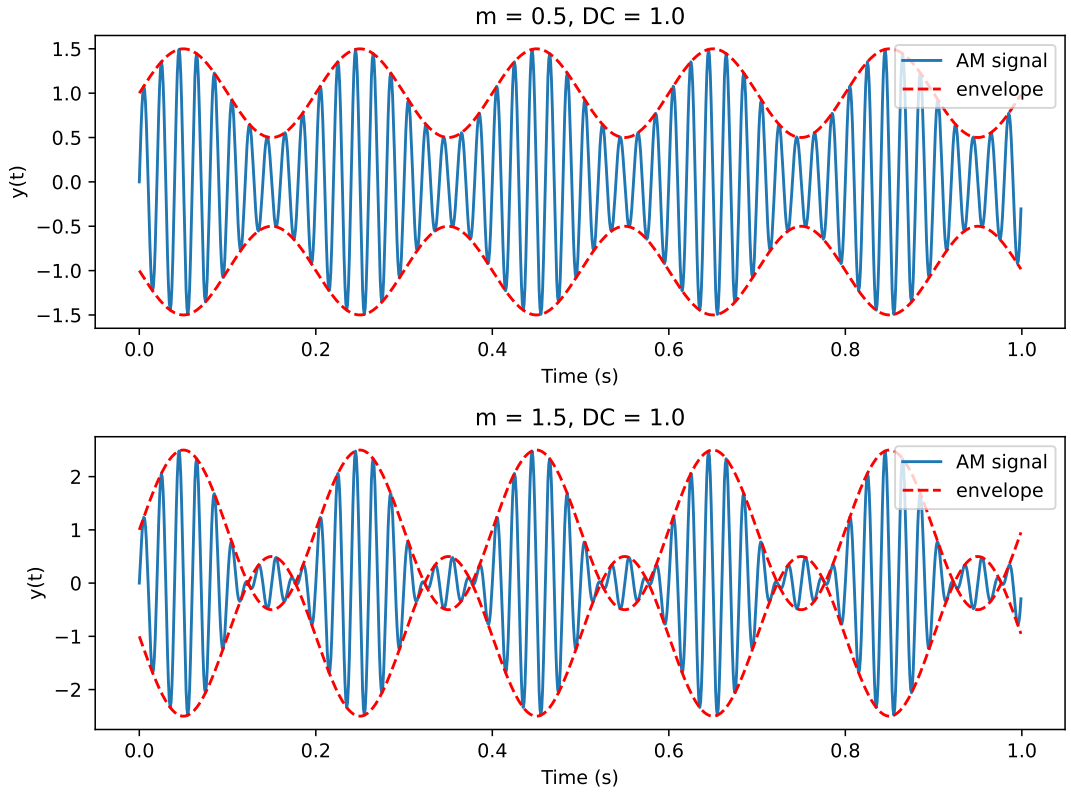


Figure 2: AM signal with $A = 1$, DC $= 1$, $\nu_m = 5$ Hz and $\nu_c = 50$ Hz. Top: $m = 0.5$. Bottom: $m = 1.5$. Red dashed lines: the envelope $\pm A[\mathrm{DC} + m\sin(2\pi\nu_m t)]$.

3.4 Why the plots look like this

- **Top, $m = 0.5$.** The fast oscillation is the carrier, with 50 cycles in 1 s. Its peaks rise and fall between $A(\mathrm{DC} - m) = 0.5$ and $A(\mathrm{DC} + m) = 1.5$, five times per second, which is the message frequency ν_m. The red dashed envelope touches every peak, so the outline of the signal is an exact copy of the 5 Hz message, shifted up by the DC offset. At $t = 0$ the envelope starts at $A \cdot \mathrm{DC} = 1$ (because $\sin 0 = 0$), while the signal itself starts at 0 because the carrier is a sine.
- **Bottom, $m = 1.5$.** Now $m > \mathrm{DC}$, so the bracket swings from $\mathrm{DC} - m = -0.5$ to $\mathrm{DC} + m = 2.5$. It is negative whenever $\sin(2\pi\nu_m t) < -\mathrm{DC}/m = -2/3$, which is about a quarter of each message cycle, centred on $t = 0.15$ s, 0.35 s, and so on. There the red dashed curves cross through zero and swap sides, and the carrier is multiplied by a negative number, so it is flipped upside down. The outline of the signal therefore has large lobes (up to 2.5) separated by small lobes (up to 0.5): it follows the size of the bracket rather than the message. A receiver that follows the peaks would recover a distorted message, so the information is damaged. This is why m must not exceed DC.

4 Task 3: Discrete Fourier Transform (DFT)

4.1 Theory

The DFT. The DFT of the N samples f_n is

$$F_k = \frac{1}{N}\sum_{n=0}^{N-1} f_n\, e^{-i2\pi kn/N}, \qquad k = 0, 1, \ldots, N-1. \tag{8}$$

The exponential can be written as $e^{-i2\pi kn/N} = \cos(2\pi kn/N) - i\sin(2\pi kn/N)$. For a given k, these are a cosine and a sine that complete exactly k cycles over the N samples. The sum multiplies every sample by them and adds the results, so it compares the signal with that frequency. If the signal contains that frequency, the products mostly have the same sign and add up to a large $|F_k|$. If it does not, positive and negative products cancel and $|F_k|$ is close to zero. The real part of F_k measures the cosine content and the imaginary part the sine content, so the magnitude $|F_k| = \sqrt{\mathrm{Re}(F_k)^2 + \mathrm{Im}(F_k)^2}$ gives the size of that frequency component whatever its phase.

Scaling the axis to hertz. Bin k completes k cycles in the record length $T = N/f_\mathrm{s}$, so its frequency is

$$\nu_k = \frac{k}{T} = \frac{k\, f_\mathrm{s}}{N}, \qquad \Delta f = \frac{f_\mathrm{s}}{N} = \frac{1}{T}. \tag{9}$$

Δf is the spacing between bins, called the frequency resolution. Two tones closer together than Δf cannot be told apart, so to resolve a spacing Δf the record must last at least $T = 1/\Delta f$. Only the first half of the array ($k < N/2$, up to f_N) holds the real frequencies. For a real signal, the second half is a mirror image of the first ($|F_{N-k}| = |F_k|$) and represents negative frequencies.

Height of a peak. Because of the $1/N$ in Equation (8), a tone of amplitude B that completes a whole number of cycles in the record lands exactly in one bin with $|F_k| = B/2$. The other half of its amplitude is in the mirrored bin in the second half of the array.

Expected spectrum of the AM signal. Expanding Equation (6) with the identity

$$\sin\theta_1 \sin\theta_2 = \tfrac{1}{2}\cos(\theta_1 - \theta_2) - \tfrac{1}{2}\cos(\theta_1 + \theta_2)$$

gives

$$y(t) = \underbrace{A\,\mathrm{DC}\sin(2\pi\nu_c t)}_{\text{carrier}} + \underbrace{\frac{Am}{2}\cos\big(2\pi(\nu_c - \nu_m)t\big)}_{\text{lower sideband}} - \underbrace{\frac{Am}{2}\cos\big(2\pi(\nu_c + \nu_m)t\big)}_{\text{upper sideband}}. \tag{10}$$

The AM signal is therefore just three pure tones: the carrier at ν_c and two sidebands at $\nu_c \pm \nu_m$. The message is carried by the sidebands, and the signal occupies the band from $\nu_c - \nu_m$ to $\nu_c + \nu_m$, a bandwidth of $2\nu_m$. Combining this with the peak height rule gives the predicted DFT peaks for the baseline in Table 3.

Table 3: Predicted DFT peaks for the baseline ($A = 1$, DC $= 1$, $m = 0.5$, $\nu_m = 5$ Hz, $\nu_c = 50$ Hz), and the values read from Figure 3.

Component	Frequency (Hz)	Amplitude in $y(t)$	Predicted $\lvert F_k\rvert$	Measured $\lvert F_k\rvert$
Lower sideband	$\nu_c - \nu_m = 45$	$Am/2 = 0.25$	0.125	0.125
Carrier	$\nu_c = 50$	$A \cdot \mathrm{DC} = 1$	0.5	0.500
Upper sideband	$\nu_c + \nu_m = 55$	$Am/2 = 0.25$	0.125	0.125

Leakage. If a tone does not complete a whole number of cycles in the record, its frequency falls between two bins. The DFT effectively treats the N samples as one period of a signal that repeats forever. With a non-whole number of cycles, the end of the record does not join smoothly onto the start, and extra frequencies are needed to describe that jump. The tone leaks into its neighbouring bins: its peak becomes lower and wider and a low "skirt" appears either side. A window function (for example a Hann window) applied to the samples before the DFT can reduce this.

Aliasing in the spectrum. As in Task 1, a component above f_{N} cannot be held by the samples. It appears folded back, at $f_{\mathrm{s}} - \nu$.

The FFT. The fast Fourier transform computes exactly the same values F_k as Equation (8), using a much faster algorithm.

4.2 Code and how it works

Listing 7: Task 3, lines 1–14: the hand-written DFT.

```
1  """-----TASK 3-----"""
2  import time
3
4  def my_dft(f):
5      """
6      DFT exactly as written on the lab sheet:
7          F_k = (1/N) * sum_{n=0}^{N-1} f_n * exp(-i*2*pi*k*n/N)
8      """
9      N = len(f)
10     n = np.arange(N)                          # all sample indices at once
11     F = np.zeros(N, dtype=complex)            # output is complex
12     for k in range(N):                        # one frequency bin per loop
13         F[k] = np.sum(f * np.exp(-1j * 2 * np.pi * k * n / N)) / N
14     return F
```

- **Line 2** imports the `time` module, used at the end to time the two DFT methods.
- **Line 9** finds the number of samples N.
- **Line 10** makes the array of all sample indices $[0, 1, \ldots, N-1]$: the n in Equation (8).
- **Line 11** creates an array of N zeros, ready to hold the results. `dtype=complex` is needed because the values F_k are complex numbers.
- **Line 12** loops over $k = 0, 1, \ldots, N-1$: one pass per frequency bin.
- **Line 13** is Equation (8) for one value of k. `np.exp(-1j * 2 * np.pi * k * n / N)` builds the complex exponential for every n at once (`1j` is Python's imaginary unit i). `f * ...` multiplies each sample by its exponential, `np.sum` adds the N products (the Σ), and `/ N` applies the $1/N$. Each pass handles all N samples in one go, so the only loop is over k.
- **Line 14** returns the full array of N values F_k.

Listing 8: Task 3, lines 17–31: scaling the axis to hertz and plotting the spectrum.

```
17 def freq_axis(N, fs):
18     """Bin index k -> Hz:  x(k) = k / (fs^-1 * N) = k * fs / N."""
19     return np.arange(N) * fs / N
```

```
20
21
22 def plot_spectrum(ax, y, fs, title):
23     """Compute my_dft of y and plot |F_k| against real frequency."""
24     N = len(y)
25     F = my_dft(y)
26     freqs = freq_axis(N, fs)
27     half = N // 2                              # first half of the array = the real
       ↪ frequencies
28     ax.plot(freqs[:half], np.abs(F[:half]))
29     ax.set_title(title)
30     ax.set_xlabel("Frequency (Hz)")
31     ax.set_ylabel("|F_k|")
```

- **Line 19** is Equation (9): the bin numbers k multiplied by f_s/N give the frequency of every bin in hertz. (The comment writes the same thing as $x(k) = k/(f_s^{-1}N)$.) This is the step that scales the axis to real units.
- **Lines 22–31** define `plot_spectrum`, which computes the DFT of a signal and its frequency axis and plots one against the other. `N // 2` is integer division (for example, `1037 // 2` is 518), and the slices `freqs[:half]` and `F[:half]` keep elements 0 to $half - 1$, which is the first half of the array, up to f_N. `np.abs(F)` gives the magnitude $|F_k|$ of each complex value.

Listing 9: Task 3, lines 34–39: the baseline spectrum.

```
34 # --- 1. Baseline spectrum using my own DFT ---
35 t, y = am_signal(A, DC, m, nu_m, nu_c, fs, N)
36 fig, ax = plt.subplots(figsize=(8, 3))
37 plot_spectrum(ax, y, fs, f"Baseline: fs={fs} Hz, N={N}, nu_c={nu_c} Hz, nu_m={nu_m} Hz")
38 ax.set_xlim(0, 100)                           # zoom in around the carrier
39 plt.show()
```

- **Line 35** generates the baseline AM signal from Task 2.
- **Lines 36–39** make a wide, short plot of its spectrum and zoom the x-axis to 0–100 Hz, because nothing appears above that.

Listing 10: Task 3, lines 41–56: exploring one change at a time.

```
41 # --- 2. Explore: change one thing at a time ---
42 cases = [
43     # (fs,   N,    nu_c, nu_m, description)
44     (1000, 1000,  50,   5,   "baseline"),
45     (1000, 200,   50,   5,   "shorter array, N = 200"),
46     (1000, 1037,  50,   5,   "N = 1037 (not a whole number of cycles)"),
47     (200,  200,   50,   5,   "lower sample frequency, fs = 200 Hz"),
48     (1000, 1000,  600,  5,   "carrier above fs/2, nu_c = 600 Hz"),
49     (1000, 1000,  50,   20,  "higher modulation frequency, nu_m = 20 Hz"),
50 ]
51 fig, axes = plt.subplots(len(cases), 1, figsize=(8, 16))
52 for ax, (fs_i, N_i, nuc_i, num_i, desc) in zip(axes, cases):
53     _, y_i = am_signal(A, DC, m, num_i, nuc_i, fs_i, N_i)
54     plot_spectrum(ax, y_i, fs_i, desc)
55 plt.tight_layout()
56 plt.show()
```

- **Lines 42–50** list six cases as $(f_s, N, \nu_c, \nu_m, \text{description})$. The first is the baseline and each of the others changes one thing, so any difference in its plot is caused by that change. In the

fourth case N is lowered together with f_s so that T, and therefore Δf, stays the same, and the only thing that changes is f_N.

- **Lines 51–56** create six stacked plots and fill each one with `plot_spectrum`. On line 53, `_` is a throwaway name for the time array, which is not needed here.

Listing 11: Task 3, lines 58–68: comparing the hand-written DFT with NumPy's FFT.

```
58 # --- 3. Compare my DFT with NumPy's FFT ---
59 start = time.perf_counter()
60 F_mine = my_dft(y)
61 t_mine = time.perf_counter() - start
62
63 start = time.perf_counter()
64 F_fft = np.fft.fft(y) / N       # NumPy's FFT has no 1/N factor, so divide to match the lab's formula
65 t_fft = time.perf_counter() - start
66
67 print("Same result?", np.allclose(F_mine, F_fft))
68 print(f"my_dft: {t_mine*1e3:.2f} ms    np.fft.fft: {t_fft*1e3:.3f} ms")
```

- **Lines 59–61 and 63–65** time each method. `time.perf_counter()` reads a precise clock before and after the calculation, and the difference is the time taken in seconds. Both methods use the baseline signal.
- **Line 64** calls NumPy's FFT. It does not include the $1/N$ factor of Equation (8), so the result is divided by `N` to make the two directly comparable.
- **Line 67** uses `np.allclose` to check that every one of the N values agrees between the two methods, to within a tiny tolerance that allows for rounding.
- **Line 68** prints both times in milliseconds (`* 1e3`).

4.3 Results

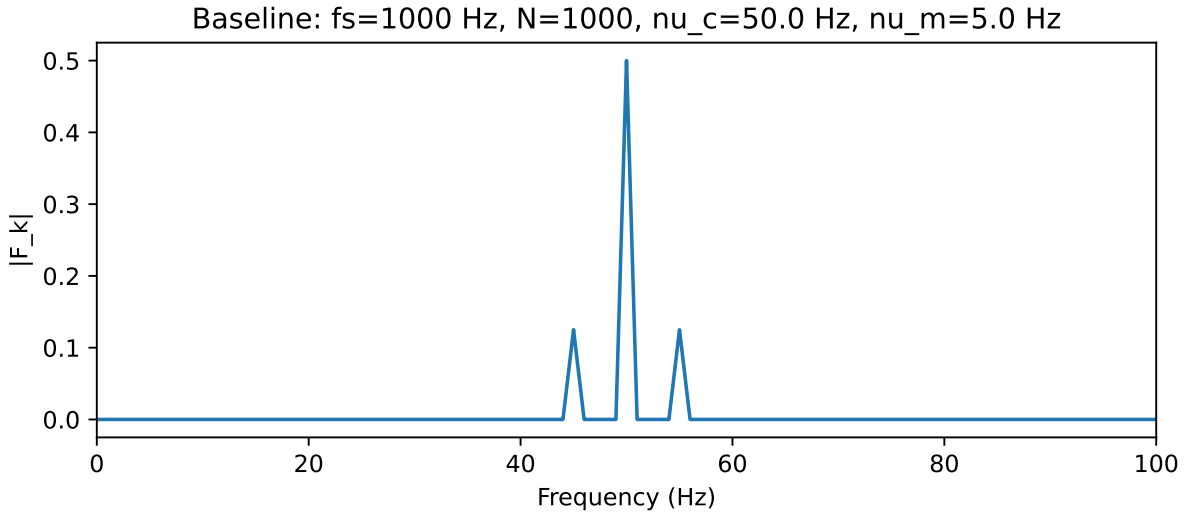


Figure 3: Magnitude of the hand-written DFT of the baseline AM signal ($f_s = 1000$ Hz, $N = 1000$, $\nu_c = 50$ Hz, $\nu_m = 5$ Hz), zoomed to 0–100 Hz.

Output of lines 67–68 (one run):

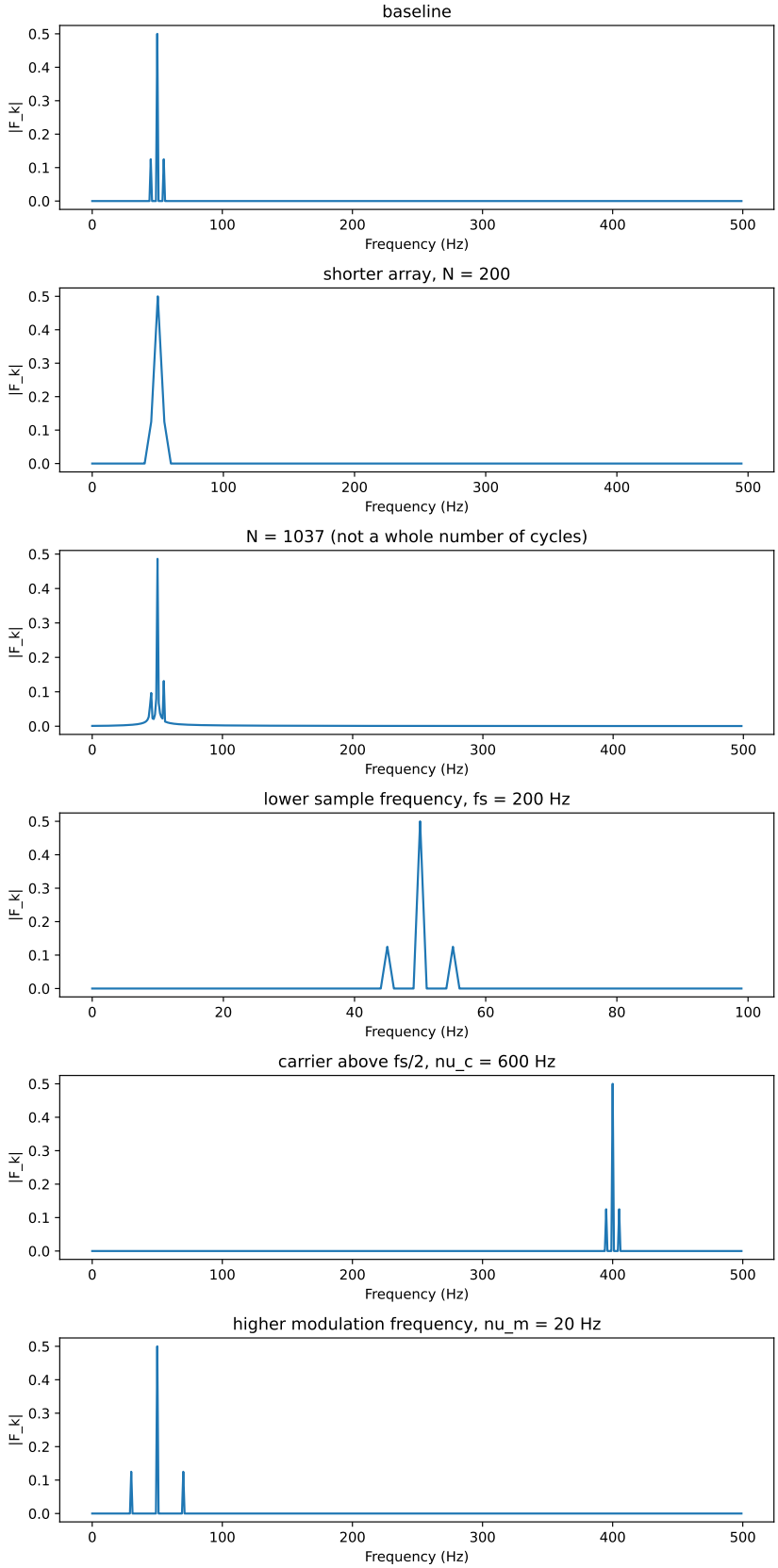


Figure 4: DFT magnitude for six cases, labelled (a) to (f) from top to bottom: (a) baseline; (b) $N = 200$; (c) $N = 1037$; (d) $f_\mathrm{s} = 200$ Hz with $N = 200$; (e) $\nu_c = 600$ Hz; (f) $\nu_m = 20$ Hz. Every other value is as in the baseline.

```
Same result? True
my_dft: 38.78 ms    np.fft.fft: 3.778 ms
```

4.4 Why the plots look like this

Baseline (Figure 3). There are three sharp peaks at exactly 45, 50 and 55 Hz, as Equation (10) predicts, and their heights of 0.125, 0.5 and 0.125 match Table 3 exactly. Every other bin is zero, apart from rounding. This is because each tone completes a whole number of cycles in $T = 1$ s (45, 50 and 55 cycles), so each one lands exactly on a bin, with $\Delta f = 1$ Hz. The peaks look like narrow triangles only because the plot joins neighbouring 1 Hz bins with straight lines. The ratio of sideband to carrier height is $0.125/0.5 = m/(2\,\mathrm{DC}) = 0.25$, so the sidebands grow with the modulation depth.

Exploring the parameters (Figure 4). Table 4 summarises the six cases.

Table 4: Summary of the six cases in Figure 4.

Plot	Change from baseline	$\Delta f = f_\mathrm{s}/N$	$f_\mathrm{N} = f_\mathrm{s}/2$	Peaks seen
(a)	none	1 Hz	500 Hz	45, 50, 55 Hz
(b)	$N = 200$	5 Hz	500 Hz	45, 50, 55 Hz (5 Hz grid)
(c)	$N = 1037$	0.964 Hz	500 Hz	near 45, 50, 55 Hz, leaking
(d)	$f_\mathrm{s} = 200$ Hz, $N = 200$	1 Hz	100 Hz	45, 50, 55 Hz
(e)	$\nu_c = 600$ Hz	1 Hz	500 Hz	395, 400, 405 Hz (aliased)
(f)	$\nu_m = 20$ Hz	1 Hz	500 Hz	30, 50, 70 Hz

- **(a) Baseline.** This is the same spectrum as Figure 3, but over the full axis. The axis ends at 500 Hz, which is f_N, because only the first half of the array is plotted. Nothing appears except the cluster around 50 Hz.
- **(b) Shorter array, $N = 200$.** The record lasts only $T = 0.2$ s, so $\Delta f = 5$ Hz and there is a bin only every 5 Hz. The three tones complete 9, 10 and 11 cycles in 0.2 s, so they still land exactly on bins with the same heights. However, they now sit in neighbouring bins, and the joined line shows a single broad peak with shoulders. A shorter record gives a coarser frequency resolution. If ν_m were below 5 Hz, the sidebands would fall in the same bin as the carrier and could not be seen at all.
- **(c) $N = 1037$.** The record lasts 1.037 s, so $\Delta f \approx 0.964$ Hz, and the tones no longer complete a whole number of cycles (the carrier completes 51.85). Measured in bins ($\nu N/f_\mathrm{s}$), the three tones sit at 46.67, 51.85 and 57.04. The lower sideband is therefore about a third of a bin from the nearest bin, the carrier 0.15 of a bin, and the upper sideband almost exactly on one. The peaks drop accordingly: the carrier reaches 0.49 instead of 0.5, the lower sideband 0.10 and the upper sideband 0.13. The missing height has leaked into the neighbouring bins (0.08 at 49.2 Hz and 0.07 at 51.1 Hz), and a low skirt spreads across the spectrum (still about 0.004 to 0.005 at 30 Hz and 70 Hz). This is leakage. The lower sideband, being furthest from a bin, loses the most. (The upper sideband is slightly above 0.125 because some of the carrier's leakage adds to it.)
- **(d) Lower sample frequency, $f_\mathrm{s} = 200$ Hz.** T is still 1 s, so Δf is still 1 Hz, but f_N is now 100 Hz and the axis stops there. The highest frequency in the signal is 55 Hz, below 100 Hz, so the spectrum is identical to the baseline: nothing is lost by sampling five times more slowly.

For an AM signal the condition is that the upper sideband, $\nu_c + \nu_m$, is below f_N, not just the carrier or the message.

- **(e) Carrier above $f_s/2$, $\nu_c = 600$ Hz.** The carrier is above $f_N = 500$ Hz, so it cannot be held by the samples. It folds back to $1000 - 600 = 400$ Hz, and the sidebands at 595 and 605 Hz fold to 405 and 395 Hz, so they swap sides. From the samples alone, this is indistinguishable from a genuine AM signal on a 400 Hz carrier. It is the frequency-domain view of the aliasing seen in Task 1(c).
- **(f) Higher modulation frequency, $\nu_m = 20$ Hz.** The sidebands move out to 30 and 70 Hz, so their distance from the carrier equals ν_m. Their heights do not change, because from Equation (10) they depend only on A, DC and m. The signal now occupies $2\nu_m = 40$ Hz of bandwidth instead of 10 Hz: a faster message needs more bandwidth.

Hand-written DFT compared with the FFT.

- **Same result.** `np.allclose` returned `True`: all 1000 values agree, and the largest difference is about 4×10^{-14}, which is rounding error. This confirms that `my_dft` implements Equation (8) correctly, including the $1/N$ scaling.
- **Speed.** In the run shown, `my_dft` took 38.8 ms and the FFT 3.8 ms. The first FFT call also includes some one-off set-up inside NumPy. Running the cell again gives about 0.02 to 0.1 ms for the FFT, while `my_dft` stays at about 35 to 40 ms, so the FFT is hundreds to over a thousand times faster. The exact times depend on the computer.
- **Why.** `my_dft` makes N passes, each with N multiplications and freshly calculated exponentials, so it needs $N^2 = 10^6$ complex multiplications for $N = 1000$. The FFT splits the sum into halves (even- and odd-numbered samples) again and again and reuses the results that different bins share, so it needs only about $N \log_2 N \approx 10^4$ operations. The gap grows with N: doubling N roughly quadruples the time for `my_dft` but only slightly more than doubles it for the FFT.

5 Task 4: Signal Filtering

5.1 Theory: filtering by multiplying in k-space

A filter is described by its frequency response, $H(f)$, which says how much of each frequency it lets through. In the frequency domain, filtering is a multiplication:

$$Y_{\text{out}}(f) = Y(f)\, H(f). \tag{11}$$

This is the convolution theorem: multiplying in the frequency domain is the same as convolving the signal with the filter's response in the time domain, but the multiplication is much simpler to compute and to design.

A square band-pass filter keeps one band of frequencies and removes everything else:

$$H(f) = \begin{cases} 1, & f_{\text{low}} \le |f| \le f_{\text{high}}, \\ 0, & \text{otherwise.} \end{cases} \tag{12}$$

The $|f|$ matters. As Task 3 showed, the spectrum of a real signal has a positive-frequency half and a mirrored negative-frequency half, and each real tone is a matching pair with one member in each half. The filter must treat both halves in the same way. Otherwise the inverse FFT gives a complex result, which is not a physical signal.

Using Equation (10), the output of each pass band can be predicted before running the code (Table 5).

Table 5: Predicted output of the three pass bands for the baseline AM signal.

Pass band	Components kept	Predicted output	Amplitude range
48–52 Hz	50 Hz	$\sin(2\pi \cdot 50t)$	constant, 1
40–60 Hz	45, 50, 55 Hz	the original $y(t)$	0.5 to 1.5
48–60 Hz	50, 55 Hz	$\sin(2\pi \cdot 50t) - 0.25\cos(2\pi \cdot 55t)$	0.75 to 1.25

5.2 Code and how it works

Listing 12: Task 4, lines 1–14: the square band-pass filter.

```
1  """-----TASK 4-----"""
2  def bandpass(y, fs, f_low, f_high):
3      """
4      Square bandpass filter applied by multiplying in k-space.
5      Keeps components with f_low <= |frequency| <= f_high and zeros the rest.
6      """
7      N = len(y)
8      Y = np.fft.fft(y)                          # to the frequency domain
9      freqs = np.fft.fftfreq(N, d=1/fs)          # signed frequency (Hz) of each bin
10     # Pass BOTH +f and -f, otherwise the result of the IFFT isn't a real signal
11     H = ((np.abs(freqs) >= f_low) & (np.abs(freqs) <= f_high)).astype(float)
12     y_filt = np.fft.ifft(Y * H).real           # multiply by the filter, then back to time
13     # .real just drops tiny imaginary rounding errors (~1e-16)
14     return y_filt, freqs, Y, H
```

- **Line 8** takes the FFT of the signal to move to the frequency domain (this is the same FFT as Task 3, without the $1/N$).
- **Line 9** gives the frequency of every bin, with its sign. For $N = 1000$ and $f_s = 1000$ Hz it returns $[0, 1, \ldots, 499, -500, \ldots, -1]$ Hz, so the second half of the array is labelled as the negative frequencies. The argument `d=1/fs` is the time between samples.
- **Line 11** builds H from Equation (12). The two comparisons test every bin at once and give arrays of `True`/`False` values; `&` combines them so that a bin is `True` only if it passes both tests, that is, if it lies inside the band at either sign. `.astype(float)` turns `True`/`False` into 1.0/0.0. The result is an array of 1s inside the band and 0s outside: the square filter.
- **Line 12** is Equation (11). `Y * H` multiplies the spectrum by the filter bin by bin, which sets every bin outside the band to zero. `np.fft.ifft` then returns to the time domain. Because the filter is symmetric, the result is real apart from rounding (the largest imaginary part here is about 2×10^{-16}), so `.real` keeps the real part.
- **Line 14** returns the filtered signal, along with the frequency axis, spectrum and filter so they can be plotted.

Listing 13: Task 4, lines 17–45: applying three pass bands and plotting the results.

```
17 t, y = am_signal(A, DC, m, nu_m, nu_c, fs, N)   # baseline signal from Task 2
18
19 filters = [(48, 52, "carrier only"),
20            (40, 60, "carrier + both sidebands"),
21            (48, 60, "carrier + upper sideband")]
```

```
22
23 fig, axes = plt.subplots(len(filters), 2, figsize=(12, 9))
24 for row, (f_low, f_high, desc) in zip(axes, filters):
25     y_filt, freqs, Y, H = bandpass(y, fs, f_low, f_high)
26     pos = freqs >= 0                              # plot positive frequencies only
27
28     # Left: spectrum with the filter window drawn over it
29     row[0].plot(freqs[pos], np.abs(Y[pos]) / N, label="|FFT| / N")
30     row[0].plot(freqs[pos], H[pos] * np.abs(Y).max() / N, "r--", label="filter")
31     row[0].set_xlim(0, 100)
32     row[0].set_title(f"Pass band {f_low}-{f_high} Hz ({desc})")
33     row[0].set_xlabel("Frequency (Hz)")
34     row[0].legend()
35
36     # Right: original vs filtered time series
37     row[1].plot(t, y, color="lightgrey", label="original")
38     row[1].plot(t, y_filt, label="filtered")
39     row[1].set_xlim(0, 0.4)                       # zoom to two message cycles
40     row[1].set_title("Time series after IFFT")
41     row[1].set_xlabel("Time (s)")
42     row[1].legend(loc="upper right")
43
44 plt.tight_layout()
45 plt.show()
```

- **Line 17** generates the baseline AM signal from Task 2.
- **Lines 19–21** list three pass bands as $(f_{\text{low}}, f_{\text{high}}, \text{label})$. They are chosen to keep the carrier only, all three components, or the carrier and upper sideband (Table 5), which changes the bandwidth from 4 to 20 to 12 Hz.
- **Line 23** creates a grid of plots with one row per pass band: the left column (`row[0]`) shows the spectrum and the right column (`row[1]`) the time series.
- **Line 25** applies the filter.
- **Line 26** makes a mask that is `True` for the positive-frequency half, so only that half is plotted.
- **Line 29** plots the spectrum divided by N, which gives the same scaling as Task 3 (peaks of 0.5 and 0.125).
- **Line 30** draws the filter. H is only 0 or 1, so it is multiplied by the tallest peak to draw it at the same height as the spectrum.
- **Lines 31–34** zoom to 0–100 Hz and add a title, label and legend.
- **Lines 37–42** plot the original signal in grey and the filtered signal on top, zoomed to the first 0.4 s (two message cycles) so the individual carrier cycles can be seen.

5.3 Results

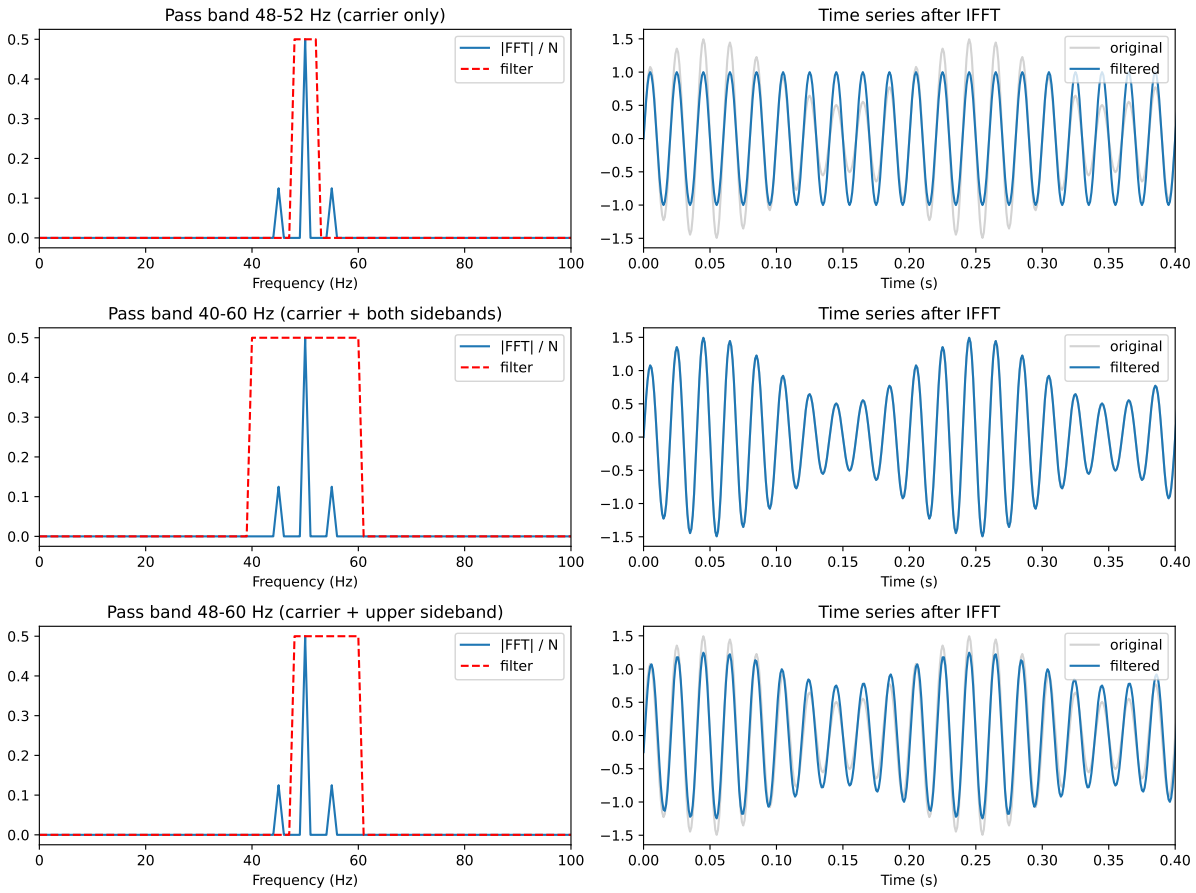


Figure 5: Square band-pass filtering of the baseline AM signal. Left: spectrum ($|\mathrm{FFT}|/N$) with the filter drawn as a red dashed line. Right: the original signal (grey) and the filtered signal after the inverse FFT (blue). Rows, top to bottom: pass bands of 48–52 Hz, 40–60 Hz and 48–60 Hz.

5.4 Why the plots look like this

- **Top row, 48–52 Hz (carrier only).** Only the 50 Hz bin lies inside the window, so both sidebands are removed. The output is a pure 50 Hz sine of constant amplitude $A \cdot \mathrm{DC} = 1$: the rise and fall of the envelope has disappeared, so the message has been removed. The carrier on its own carries no information; the message is entirely in the sidebands.
- **Middle row, 40–60 Hz (all three components).** Every component lies inside the window, so the output is identical to the original (the largest difference is about 4×10^{-14}) and the filtered line hides the grey one. A pass band at least $2\nu_m = 10$ Hz wide and centred on ν_c is enough to pass the whole AM signal.
- **Bottom row, 48–60 Hz (carrier and upper sideband).** The lower sideband is removed, leaving $\sin(2\pi \cdot 50t) - 0.25\cos(2\pi \cdot 55t)$. The two remaining tones are 5 Hz apart, so they drift into and out of step five times per second. The amplitude therefore still rises and falls at the message frequency, but only between 0.75 and 1.25 (measured 0.753 to 1.247) instead of 0.5 and 1.5. That is half the original swing, because half of the sideband content has been removed. The message is still present in a single sideband, at half the strength.
- **Shape of the filter.** The red dashed filter has sloping sides only because the plot joins points spaced 1 Hz apart. The filter itself jumps straight from 0 to 1 between neighbouring bins.

- **Why the cut is clean.** Each tone sits exactly in one bin (a whole number of cycles in 1 s), so a square filter either keeps it completely or removes it completely. If the signal leaked, as in Task 3(c), part of the spread-out energy would lie outside the pass band and be cut off too, so even a band containing all three tones would not return an exact copy.

6 Conclusion

All four tasks were completed, and every plot agrees with what the theory predicts.

- **Sampling (Task 1).** The sample frequency must be more than twice the signal frequency. At $f_\mathrm{s} = 8$ Hz the samples of the 5 Hz wave were identical to those of a 3 Hz wave, which is aliasing. The number of samples sets the record length $T = N/f_\mathrm{s}$, not the detail within each cycle.
- **Amplitude modulation (Task 2).** AM is the product of the message bracket and the carrier. The DC offset keeps the bracket positive, and with $m \leq \mathrm{DC}$ the envelope is an exact copy of the message. With $m > \mathrm{DC}$ the carrier flips and the envelope no longer matches the message.
- **DFT (Task 3).** The hand-written DFT produced the carrier at ν_c and sidebands at $\nu_c \pm \nu_m$, with the predicted heights $A \cdot \mathrm{DC}/2$ and $Am/4$, on an axis scaled with $\nu_k = kf_\mathrm{s}/N$. N sets the resolution $\Delta f = f_\mathrm{s}/N$; a non-whole number of cycles causes leakage; f_s sets f_N, and components above it alias; ν_c moves the whole cluster; and ν_m sets the sideband spacing and the bandwidth $2\nu_m$. NumPy's FFT gives the same values to about 10^{-14} and is hundreds to thousands of times faster.
- **Filtering (Task 4).** Multiplying the spectrum by a square window and taking the inverse FFT filters the signal. Removing the sidebands removed the message, passing the full $2\nu_m$ band returned the original signal, and keeping one sideband kept the message at half strength.

Possible improvements:

- apply a window (for example a Hann window) before the DFT to reduce leakage when the record does not contain a whole number of cycles;
- plot $|F_k|$ in dB so that small components, such as the leakage skirt, are visible;
- time both DFT methods for several values of N to show directly how their run times grow;
- use one name for the sample frequency throughout the code (`sf` in Task 1 and `fs` afterwards).